

Sensitive Data Exposure Task 4: -

Sensitive Data Exposure occurs when a website doesn't properly protect (or remove)

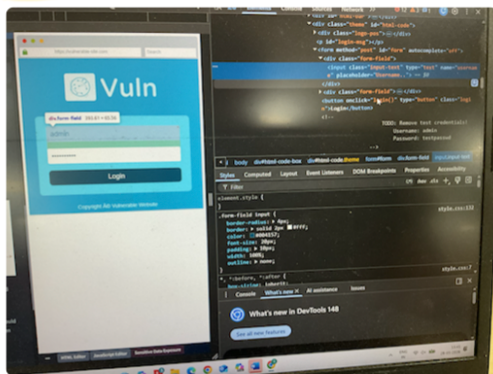
Sensitive client information to end users. usually found in site's frontend source code.

As we know websites are built using many HTML elements (tags), all of which we can see simply by "viewing page source". A website developer may have forgotten to remove login credentials, hidden links to private parts of website or other sensitive data shown in HTML or JavaScript.

Sensitive information can be potentially leveraged to further attacker's access within different parts of web application. Ex:- there could be HTML comments with temporary login credentials.

And if you view page source code and found this, you could use this login credentials to log in elsewhere on application (or worse, used to access other backend components of site).

Whenever you're assessing web application for security its url, one of first thing you should do is review page source code to see if you can find any exposed login credentials or hidden files.



In comments found username and password

this is a Sensitive Data Exposure.

Quiz:-

1) View website. what is password hidden in source code?

A) testPassword

Task 5: - HTML Injection: -

HTML injection is vulnerability that occurs when unfiltered user input is displayed on page.

If a website fails to sanitise user input (filter any "malicious" text that user inputs into website).

and that input is used on page, an attacker can inject HTML code into vulnerable website.

Input sanitisation is very important in keeping website secure, an information a user

inputs into website is often used in frontend and backend functionality. In another

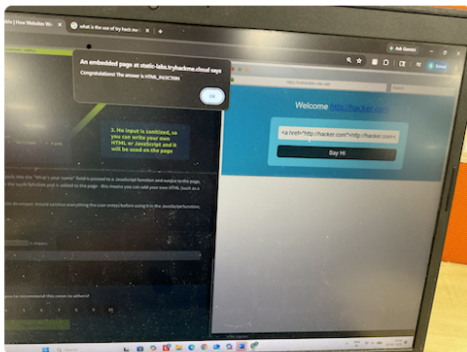
As vulnerability you'll explore is database injection. Now let's focus on HTML Injection (Client side).

When user has control of how their input is displayed, they can submit HTML (or JS) code, and browser will use it on page, allowing user to control page appearance and functionality.

The general rule is never trust user input. To prevent malicious input, the website developer should sanitise everything the user enters before using it in JS function; In this case the developer could remove any html tag.

Quiz:-

View website on this task and inject HTML so that malicious link to <http://hacker.com> is shown.



Code:- `<a href="http://hacker.com">http://hacker.com</a>`

